Advanced Data Management
Mock Exam v2 (Chapters 1–6): Full Solutions

Every answer in the format the rubric wants: the letter or the number first, the method, the one-sentence decision, the trap; with a map to the textbook chapters

Advanced Data Management

Mock exam v2, 100 points, 2 hours, same format as the midterm

How to use this document

Mock v2 is the midterm's format: eight quick checks with a one-line justification, three problem situations that end in a decision, one real plan to annotate, and two discussion tickets out of three. Grading rewards method: a wrong final number with a correct formula earns most of the points, a bare number earns little. Every solution below is therefore written the way the review session's five steps ask: question, data, formula from the sheet, computation with units, one sentence that quotes the number. All three discussion tickets are answered so that you can choose on the day. The database is `adm_lab`: `customers` 10,000 rows / 73 pages, `products` 200 / 2, `orders` 500,000 / 3,185 pages, 8 KB pages, PostgreSQL 16. All arithmetic was checked by hand twice.

Part	Content	Chapters	Points
A	8 quick checks	1, 2, 3, 4, 5, 6	24
B	3 problem situations	4 (sort, joins), 3 and 6 (B^+-tree latency)	30
C	one real plan, 5 questions	4, 5, 6	26
D	2 of 3 tickets	6 (D1, D3), 4 (D2)	20

Part A: Quick checks (24 points)

Three points each: one for the letter, two for the justification.

Problem: A1 Which component decides Seq Scan versus Index Scan on `orders`? [3 pts]

(a) the parser (b) the planner (c) the executor (d) the buffer manager

Solution

(b) the planner. The access path is a physical-plan choice made by costing both alternatives from the statistics (sequential pages at cost 1 against matching rows at random cost 4). The parser only builds the query tree; the executor runs whatever plan it is given; the buffer manager serves page requests and never sees the SQL.

Problem: A2 Delete half of `orders` at random, run a normal `VACUUM`. Pages in the file? [3 pts]

(a) about 1,592 (b) about 3,185 (c) about 6,370 (d) 0

Solution

(b) about 3,185. A plain `VACUUM` marks the dead tuples' space as free inside each page and records it in the free-space map; it does not move live rows between pages, so it can only truncate *trailing* pages that became completely empty. With random deletes every page keeps about half its rows, so no page empties and the file stays at 3,185 pages, each half full. Only `VACUUM FULL` or `CLUSTER` rewrites the table into about 250,000/157 $\approx$ 1,592 pages. Trap: (a) is what `VACUUM FULL` gives; (c) is a doubling nobody asked for.

Problem: A3 Index on `orders(cust_id, order_date)`: which query uses it best? [3 pts]

(a) `WHERE order_date = '2025-03-01'` (b) `WHERE cust_id = 7 AND order_date > '2025-06-01'`
(c) `WHERE cust_id + 0 = 7` (d) `ORDER BY order_date LIMIT 10`

Solution

(b). A multi-column B^+-tree is sorted by `cust_id` first, then `order_date` within each customer. (b) descends on the equality `cust_id = 7` and then walks the contiguous range of that customer's dates: the leftmost-prefix rule used in full. (a) has no predicate on the leading column: the dates of 2025-03-01 are scattered across all 10,000 customers, so the index cannot descend (a full index scan at best). (c) wraps the column in an expression; the planner does not know that `cust_id + 0` is `cust_id`, so the index is unusable. (d) wants an order on the second column alone, which the index does not provide.

Problem: A4 Hash join of `customers` (73 pages) and `orders` (3,185 pages): build side? [3 pts]

(a) `orders`, the larger (b) `customers`, the smaller (c) either, same cost (d) both hashed

Solution

(b) `customers`. The build side is read entirely into an in-memory hash table before the first probe; the cost $B_R + B_S$ holds only while that table fits in `work_mem`. 73 pages (about 600 KB, 519 KB in the real plan of Part C) fit in one batch; 3,185 pages (25 MB) would not fit the default 4 MB and would be split into batches written to disk, a Grace hash join at $3(B_R + B_S)$. Smaller build side: one batch, each table read once. (c) is only true when *both* fit; (d) describes Grace partitioning, where both are hashed to disk, which is what we avoid.

Problem: A5 Sort 6,000 pages with $M = 100$; run generation counts as a pass. Passes? [3 pts]

(a) 1 (b) 2 (c) 3 (d) 60

Solution

(b) 2. Runs $= \lceil 6{,}000/100 \rceil = 60$. The merge fan-in is $M - 1 = 99 \geq 60$, so all runs merge in one pass: $\lceil \log_{99} 60 \rceil = 1$. Total $1 + 1 = 2$ passes, about $2 \times 6{,}000 \times 2 = 24{,}000$ page I/Os. Trap: (d) 60 is the number of runs; (a) would need the file to fit in memory ($M \geq 6{,}000$).

Problem: A6 `amount` uniform on 5–105, 200 distinct `product_id`. Rows for `amount > 95 AND product_id = 7`? [3 pts]

(a) about 250 (b) about 2,500 (c) about 50,000 (d) about 52,500

Solution

(a) about 250. Range: $(105 - 95)/(105 - 5) = 0.10$. Equality: $1/n_{\text{distinct}} = 1/200 = 0.005$. AND under independence: $0.10 \times 0.005 = 0.0005$; $0.0005 \times 500{,}000 = 250$. Traps: (c) is the range alone; (b) is 0.10×0.05, a wrong distinct count; (d) adds the selectivities $(0.10 + 0.005)$ instead of multiplying, the classic OR/AND confusion.

Problem: A7 `Buffers: shared hit=40 read=3145`. What does `read=3145` mean? [3 pts]

(a) 3,145 pages read from the disk (b) 3,145 pages not in shared buffers; some may come from the OS cache
(c) the hit ratio is 40% (d) 3,145 rows were read

Solution

(b). `read` counts pages that missed `shared_buffers` and were requested from the operating system; the OS page cache may serve them from RAM in microseconds, and the counter cannot tell. Units are pages, not rows (d). The hit ratio of this execution is $40/(40 + 3{,}145) = 1.3\%$, not 40% (c): a cold scan of `orders` ($40 + 3{,}145 = 3{,}185$, the whole table).

Problem: A8 Steal + no-force: what must the log contain to recover after a crash? [3 pts]

(a) undo only (b) redo only (c) undo and redo (d) nothing, pages are on disk

Solution

(c) undo and redo. Steal: a dirty page of an *uncommitted* transaction may be written to disk, so after a crash it must be rolled back: undo information (old values). No-force: a committed transaction's pages need *not* be on disk at commit, so after a crash they must be re-applied: redo information (new values). Both go into the write-ahead log, written before the data page and forced at commit. (d) describes no-steal + force, the slow cell nobody uses.

Where it is in the textbooks

A1: Chapter 1, §3.4 (stage 4, optimisation into a physical plan) and Chapter 5, §5.2 (access path from the cost model). A2: Chapter 2, §3.2 (slotted pages and free space) and §6.1 (how PostgreSQL stores a table: dead tuples, VACUUM, the free-space map). A3: Chapter 3, §4.4 (multi-column indexes and the leftmost-prefix rule) and §4.3 (expression indexes: why `cust_id + 0` needs one). A4: Chapter 4, §5.4 (hash join: build side, Grace partitioning, `Batches`). A5: Chapter 4, §4.3 (counting the cost of external merge sort). A6: Chapter 5, §4.1 (selectivity as probability: range, equality, AND). A7: Chapter 6, §7.1 (anatomy of the `Buffers:` line) and §7.2 ("read" is not "disk"). A8: Chapter 6, §6.2 (steal and force, the logging matrix) and §6.3 (the write-ahead rule).

Part B: Problem situations (30 points)

Problem: B1 "Doubling `work_mem` will halve the sort time" [10 pts]

Nightly sort of `orders_archive`, 1,200 pages, with $M = 25$ buffer pages. (a) Today: runs, passes, total I/O. (b) With $M = 50$. (c) Is the DBA right? How much memory makes the sort a single pass?

Solution

Data. $B = 1{,}200$ pages; $M = 25$, then 50. **Formulas.** runs $= \lceil B/M \rceil$; merge passes $= \lceil \log_{M-1}(\text{runs}) \rceil$; total I/O $= 2B \times (1 + \text{merge passes})$.
(a) $M = 25$**.** Runs $= \lceil 1{,}200/25 \rceil = 48$ sorted runs of 25 pages. Fan-in $M - 1 = 24$: $48 \to 2 \to 1$, that is $\lceil \log_{24} 48 \rceil = 2$ merge passes. Total passes $1 + 2 = 3$; total I/O $= 2 \times 1{,}200 \times 3 = 7{,}200$ page I/Os (3,600 reads + 3,600 writes).
(b) $M = 50$**.** Runs $= \lceil 1{,}200/50 \rceil = 24$. Fan-in $49 \geq 24$: one merge pass. Total passes 2; total I/O $= 2 \times 1{,}200 \times 2 = 4{,}800$.

(c) Decision. With 25 pages the sort makes 3 passes, so 7,200 I/Os; with 50 pages it makes 2 passes, so 4,800: the I/O drops by one third, not by half. The DBA is wrong because passes are a *ceiling of a logarithm*, not proportional to memory: doubling M removed exactly one merge pass, and no further doubling (100, 200, 400 pages) removes another, since one merge pass is the minimum as long as the file does not fit. A *single* pass needs the whole file in memory, $M \geq B = 1{,}200$ pages = 9.6 MB of `work_mem`; then the sort reads once and writes nothing (I/O = 1,200, or 2,400 if the result must be written). The one-merge-pass threshold, for the record, is already $M = 36$ ($\lceil 1{,}200/36 \rceil = 34 \leq 35$), so 50 was more than needed for the gain it got.

Problem: B2 `GET /orders/{id}` must answer in less than 1 ms [10 pts]

20,000,000 rows; B$^+$-tree on `id` with about 250 entries per page. Random page read 100 μs on SSD, 10 ms on HDD; a page in shared buffers 0.1 μs. (a) Levels, and page reads per lookup including the heap page. (b) Worst case (nothing in memory) on SSD and HDD. (c) Root and second level always in shared buffers: latency on SSD, and what to buy or keep.

Solution

(a) 4 levels, 5 page reads. Levels = $\lceil \log_{250}(2 \times 10^7) \rceil$: $250^3 = 15{,}625{,}000 < 2 \times 10^7 < 250^4 = 3.9 \times 10^9$, so 4. Counting from the leaves confirms it: 20,000,000/250 = 80,000 leaf pages; 80,000/250 = 320 pages above them; 320/250 $\rightarrow$ 2 pages; then the root: 80,000 $\rightarrow$ 320 $\rightarrow$ 2 $\rightarrow$ 1, four levels. A lookup reads root, two inner pages, one leaf, then the heap page: $4 + 1 = 5$ page reads.
(b) Cold. SSD: 5×100 μs = 500 μs = 0.5 ms: the 1 ms target is met, with no margin for anything else (network, parsing, locking). HDD: 5×10 ms = 50 ms: missed by a factor of 50.
(c) Root and level 2 cached. Two hits at 0.1 μs and three reads (inner level 3, leaf, heap) at 100 μs: $0.2 + 300 \approx 300$ μs = 0.3 ms on SSD, comfortably under 1 ms. On an HDD the same three reads cost 30 ms: still 30 times over. **Decision:** keep (or buy) the SSD; an HDD cannot meet 1 ms for a 4-level tree whatever the cache, because a single random read already costs 10 ms. The next lever is RAM: the whole index is 80,322 pages $\approx$ 630 MB, so a `shared_buffers` of a few GB keeps every index level resident and leaves one SSD read, the heap page, at about 0.1 ms per lookup.

Problem: B3 "Hash joins are buggy, I switched them off" [10 pts]

`SET enable_hashjoin = off`. Join of `customers_big` ($B_R = 80$ pages) and `orders_2025` ($B_S = 3{,}200$) on `cust_id`, no useful index, $M = 10$. The planner now uses a block nested loop with R outer. (a) BNL cost. (b) Hash join cost at $M = 10$: does R fit, which variant? (c) Both at $M = 100$. (d) What do you tell the colleague?

Solution

(a) BNL, $M = 10$**.** $B_R + \lceil B_R/(M-2) \rceil \times B_S = 80 + \lceil 80/8 \rceil \times 3{,}200 = 80 + 10 \times 3{,}200 = 32{,}080$ page I/Os. The outer is read once; the inner is read ten times, once per 8-page chunk of the outer.
(b) Hash join, $M = 10$**.** R does not fit: 80 pages > 10. Grace hash join: partition both inputs to disk (read + write each), then join partition by partition (read each once): $3(B_R + B_S) = 3 \times 3{,}280 = 9{,}840$. (Feasible: 9 partitions of about 9 pages each fit in 10 buffers.) The hash join is $32{,}080/9{,}840 \approx 3.3$ times cheaper.
(c) $M = 100$**.** BNL: $80 + \lceil 80/98 \rceil \times 3{,}200 = 80 + 3{,}200 = 3{,}280$: the whole outer fits in one chunk, so S is read once. Hash: $R = 80 \leq 100$ fits, in-memory variant: $B_R + B_S = 3{,}280$. The two tie: with enough memory both algorithms read each input exactly once.
(d) Decision. The block nested loop reads S ten times at $M = 10$, so it costs 32,080 pages against 9,840 for the hash join: switching hash joins off made the report three times more expensive, and the setting applies to *every* join in the session, not just this one. Raising memory to 100 pages is what actually fixed it, and at that point the two algorithms cost the same, so the "bug" was never the algorithm. Re-enable `enable_hashjoin` (it is a planner cost penalty meant for experiments, not a production switch), keep the larger `work_mem` for this report, and if a wrong result was really observed, reproduce it with the plan and the data and file it; a planner setting is not a fix for a correctness bug.

Where it is in the textbooks

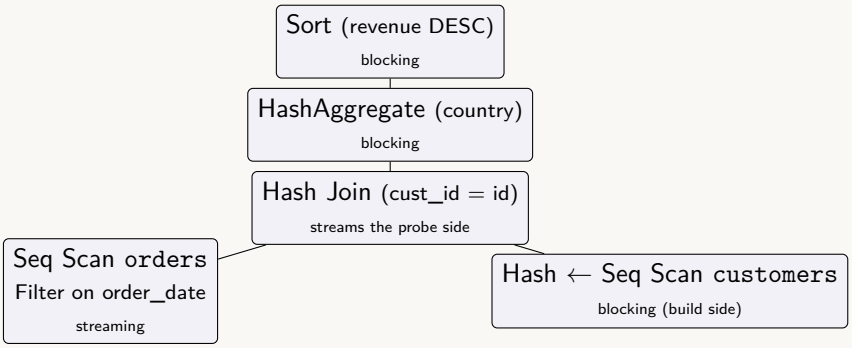
B1: Chapter 4, §4.2 (why the fan-in is $M - 1$) and §4.3 (counting the cost; the passes-versus-memory shape); classic problem P1 there. B2: Chapter 3, §3.2 (searching a B$^+$-tree, height and the $h + 1$ page reads) and §1.2 (fanout); Chapter 6, §1.3 (effective access time) and §2.1 (what is resident: the hot index levels). B3: Chapter 4, §5.2 (block nested-loop), §5.4 (hash join, in-memory versus Grace) and §5.5 (which join wins, and when memory makes them tie); §7.2 (the `enable_*` switches are penalties, not prohibitions).

Part C: Read a real plan (26 points)

Ticket REP-12, the December revenue report: `orders` joined to `customers`, filtered on `order_date >= '2025-12-01'`, grouped by country, sorted by revenue. Run 1 just after a restart (82.9 ms, `shared hit=3 read=3258`), run 2 immediately after (62.3 ms, `shared hit=3258`), run 3 with `work_mem = '64kB'` (71.5 ms, `Batches: 8`).

Problem: C1 Draw the plan as a tree; for each node, blocking or streaming? [5 pts]

Solution



Read bottom-up. **Seq Scan on `orders`**: streaming; it emits each qualifying row as it reads (42,299 kept, 457,701 removed by the filter). **Seq Scan on `customers` → Hash**: the Hash node must read *all* 10,000 customers and build the table before the join can produce its first row, so the build side is blocking (a pipeline breaker). **Hash Join**: once the table is built it streams: each `orders` row is probed and passed on immediately. **HashAggregate**: blocking; it must see every joined row before any of the 5 country totals is final. **Sort**: blocking; it needs all 5 groups to order them (top node, `quicksort`, 25 kB). Two of the five nodes stream; the plan therefore produces its first output row only at the very end, which is normal for a report with a `GROUP BY` and an `ORDER BY`.

Problem: C2 Is the estimate of the Seq Scan on `orders` good? q-error and source. [4 pts]

Solution

Yes: q-error $= \max(43{,}388/42{,}299,\ 42{,}299/43{,}388) = 1.026$, a 2.6% overestimate. Anything under 2 is good; 10 is the red flag. The estimate comes from `pg_stats` for `order_date`: the planner counts the fraction of the equi-depth histogram's buckets (and of the last bucket, by linear interpolation) lying at or above 2025-12-01, about $43{,}388/500{,}000 = 8.7\%$ of the rows, and multiplies by `reltuples` $= 500{,}000$. It is accurate because the statistics are fresh (`ANALYZE` after the inserts) and a single-column range predicate on a histogram is the easy case; the join estimate just above it (43,388 against 42,299 actual) inherits the same precision because `cust_id = id` is a key–foreign-key join: every order matches exactly one customer.

Problem: C3 Explain the `Buffers` lines of runs 1 and 2: 3,258, read → hit, and why only $1.3\times$? [6 pts]

Solution

3,258 = 3,185 + 73: every page of `orders` (its scan shows `shared read=3185`) plus every page of `customers` (`read=73`), each read exactly once by the two sequential scans. The Hash Join, HashAggregate and Sort touch no table pages; their inputs are rows already in memory (the `shared hit=3` are catalog or visibility-map pages).

Why `read` becomes `hit`. A restart empties `shared_buffers`. Run 1 misses on every page and loads it into the pool; both tables are below the ring threshold (`shared_buffers`/4 = 16,384/4 = 4,096 pages, and 3,185 < 4,096), so all 3,258 pages are kept, not cycled through a 32-page ring. Run 2 finds every page resident: 3,258 hits, zero reads, same plan, same rows.

Why 1.3× and not 1,000×. Because the 3,258 "reads" did not come from the disk. A PostgreSQL restart drops its own pool but not the operating system's page cache, which still held the two files; each read was a system call that copied a page from the OS cache, a few microseconds, about 20 ms in total: $82.9 - 62.3 = 20.6$ ms. Had the pages really come from disk, a sequential read of $3{,}258 \times 8$ KB ≈ 25 MB would have added about 50 ms at 500 MB/s, and random HDD reads $3{,}258 \times 10$ ms ≈ 33 s. Also, most of the 62 ms is CPU work that does not change between runs: filtering 500,000 rows, hashing and probing 42,299 of them, aggregating. The warm run is the floor; the cold run adds only the cost of where the pages were found.

Problem: C4 `CREATE INDEX ON orders (order_date);` correlation −0.0015. Will it help? [6 pts]

Solution

The query keeps 42,299/500,000 = 8.5% of the rows, and because the correlation is −0.0015, essentially zero, those rows are spread uniformly over the file rather than clustered in a date range. On average $42{,}299/3{,}185 \approx 13$ matching rows sit on *every* page; the chance that a page has none is $(1 - 0.085)^{157} \approx e^{-13.9} \approx 10^{-6}$. An index scan would therefore visit all 3,185 heap pages anyway, in index order, that is random order on disk: $3{,}185 \times 4 = 12{,}740$ cost units against the sequential scan's 3,185 (plus the index's own leaf pages, about 120 for 42,299 entries). Even a bitmap heap scan, which sorts the page numbers first, still reads all 3,185 pages and only saves the random-access penalty while adding the bitmap build. **No**, the index will not help this report; the planner will keep the Seq Scan, and the index will cost space and slow every insert for nothing. It *would* help if the table were physically ordered by date (correlation near 1, as after `CLUSTER orders USING` that index, or for an append-only table where dates arrive in order): then 8.5% of the rows occupy about 8.5% of the pages, ≈ 271 pages, and a BRIN index would do it at almost no cost.

Problem: C5 Run 3: what changed in the Hash node, what do `Batches: 8` and the temp lines mean, `work_mem` recommendation? [5 pts]

Solution

What changed. The hash table of `customers` needed 519 kB (run 1: `Buckets: 16384 Batches: 1 Memory Usage: 519kB`). With `work_mem = 64kB` it no longer fits, so the build is split into **8 batches** (`Buckets: 4096 Batches: 8 Memory Usage: 80kB`): a Grace hash join. The build side is partitioned by hash value into 8 partitions; the first stays in memory, the other 7 are written to temporary files (`temp written=28` on the Hash node: 7 partitions, about 4 pages each). The probe side is partitioned the same way: `orders` rows that belong to batches 2–8 are written out and read back when their batch is processed, which is the top-level `temp read=183 written=183` (183 temp pages written once, read once). The plan shape is unchanged; only memory moved to disk, and the time went from 62.3 to 71.5 ms (+15%), small here because the temp files are tiny and sit in the OS cache, large on a bigger build side.

Recommendation. `work_mem` must exceed the build side's need with margin: at least 1 MB for this report (519 kB plus overhead), and the default 4 MB is already right. If the session was lowered for some other reason, set it for this statement only: `SET LOCAL work_mem = '4MB'` inside the report's transaction. Do not raise the global value to fix one report: `work_mem` is allocated per sort or hash per connection, so a global increase multiplies across every concurrent query.

Where it is in the textbooks

C1: Chapter 4, §2.2 (pipeline breakers) and §7.1 (reading a whole plan); Chapter 1, §3.5 (the iterator model). C2: Chapter 5, §3.2 (histograms) and §4.1–4.2 (range selectivity, key–foreign-key join size); §7.5 for the q-error habit. C3: Chapter 6, §7.1–7.2 (the `Buffers:` line; "read" is not "disk": this exact ticket is the worked example) and §5.3 (the ring threshold). C4: Chapter 3, §2.2 (selectivity) and §5.1 (when an index helps: pages, not rows); Chapter 5, §5.2 (access-path cost) and §7.3 (correlation in `pg_stats`). C5: Chapter 4, §5.4 (Grace hash join, `Batches`) and §7.2 (`work_mem` experiments); Chapter 6, §7.1 (the `temp` counters are not the pool).

Part D: Discussion tickets (20 points, choose 2 of 3)

Each answer follows the rubric's shape: claim, mechanism, number, consequence. All three are answered.

Problem: D1 64 GB server, PostgreSQL only; the DBA wants `shared_buffers = 52GB` (80%) [10 pts]

What happens to the OS page cache, to sorts and hash joins, to checkpoints? What to measure first?

Solution

Claim. I disagree, because PostgreSQL uses two caches and several other pools of memory, and `shared_buffers` is only one of them; 80% starves the rest.

Mechanism, three effects. (1) *The OS page cache.* Every page that misses `shared_buffers` is read through the OS cache (the "read" of A7), WAL segments are written through it, temporary files for sorts and hashes live in it. With 12 GB left for the OS, the kernel and every other process, pool misses become real disk reads at 100 μs to 10 ms instead of microseconds, and the double-buffering that makes a cold restart cheap disappears. (2) *Sorts and hash joins.* `work_mem` is allocated *outside* `shared_buffers`, per sort or hash node, per connection. With 100 connections and a 64 MB `work_mem`, a busy moment needs 6.4 GB that no longer exists; the server swaps, or the DBA lowers `work_mem` and every report falls into external sorts and multi-batch hash joins (`Batches: 8`, run 3 of Part C). (3) *Checkpoints.* A 52 GB pool can hold tens of GB of dirty pages between checkpoints; each checkpoint must write them all within its interval, so the I/O comes in bursts every few minutes and latency spikes while it runs (and the background writer has more frames to scan).

Number. 52 GB is 6.8 million frames; the lab's whole database is 3,260 pages. Hit ratio curves plateau once the working set fits (Chapter 6 §5.4): beyond that, each extra GB caches cold pages. The usual starting point is 25%, 16 GB here, and gains above about 40% are rare.

What to measure before deciding. The per-table hit ratio from `pg_statio_user_tables` and the database-wide one from `pg_stat_database`; the distribution of `usagecount` in `pg_buffercache` (mostly 4–5: the pool is full of hot pages and more would help; mostly 0–1: it is already larger than the working set); the size of the working set, that is the tables and indexes the OLTP queries actually touch; `temp_bytes` and `Batches` in the slow reports (a `work_mem` problem, not a pool problem); and checkpoint statistics (`checkpoints_req`, `buffers_checkpoint`). Then raise `shared_buffers` in steps while the hit ratio still responds, and stop when it plateaus.

Problem: D2 `enable_nestloop = off` and `enable_mergejoin = off` for the whole database [10 pts]

One situation where each disabled algorithm is clearly the best; consequences of the change.

Solution

Claim. Hash join is not always the best: for example, a lookup that joins a handful of outer rows to an indexed inner, or a join whose inputs are already sorted, or any join that is not an equality.

Nested loop is best when the outer side is tiny and the inner has an index on the join key (index nested loop): `SELECT ... FROM customers c JOIN orders o ON o.cust_id = c.id WHERE c.id = 42`. One outer row, one index descent on `orders(cust_id)`: about $1+(3+1)$ pages for a few dozen orders, in well under a millisecond. The hash join must read the *whole* probe side, 3,185 pages of `orders`, to find the same rows: hundreds of times more I/O. The same holds for every `LIMIT`-style or "few outer rows" join, and nested loop is also the *only* algorithm for non-equi joins (`o.amount BETWEEN p.lo AND p.hi`, inequalities), which hashing cannot do at all.

Merge join is best when both inputs arrive already sorted on the join key, from index scans on `customers(id)` and `orders(cust_id)` or from an earlier sort: then it costs $B_R + B_S$ with no hash table and no memory at all, streams, and delivers output sorted on the key, so an `ORDER BY cust_id` above it is free. It is also the robust choice when the build side is far too large for `work_mem`: $\text{sort}(R) + \text{sort}(S) + B_R + B_S$ against a multi-batch Grace hash join at $3(B_R + B_S)$ with temp files.

Consequences. The `enable_*` switches do not forbid an algorithm; they add a huge cost penalty (10^{10}) to it. So every point query now reads whole tables through a hash join (the 1 ms lookup becomes a 60 ms scan), every sorted output is re-sorted after a hash join, every join builds a hash table that competes for `work_mem` and spills into batches under load, and for non-equi joins the planner still has to pick a nested loop but with absurd cost numbers that distort every plan around it. The setting is session-level in intent, for experiments on one query; applied database-wide it is a regression for the OLTP side of the workload. Reset both, and if one report was helped, find the bad estimate that made the planner choose the wrong join there (`ANALYZE`, `CREATE STATISTICS`).

Problem: D3 "COMMIT takes 2 ms but the transaction changed 300 pages; how is the data safe?" [10 pts]

Explain, then describe what happens after a power cut to a committed and to an uncommitted transaction.

Solution

Claim. At `COMMIT`, PostgreSQL only writes the transaction's *log records* to disk, not its data pages; durability lives in the write-ahead log, and the pages follow later.

Mechanism. While the transaction runs, each change dirties a page in `shared_buffers` and appends a WAL record (page id, old and new bytes) to an in-memory log buffer. `COMMIT` appends a commit record and `fsync`s the log file up to it: one *sequential* write of a few kilobytes at the end of

one file, which is why it takes 2 ms. The 300 dirty pages, 300 × 8 KB = 2.4 MB at random positions in the data files, stay in memory (no-force) and are written by the background writer or the next checkpoint, in the background, at the system's convenience. The write-ahead rule guarantees that whenever a data page does reach disk, the log describing it is already there.

Number. Forcing the 300 pages at commit would cost 300 random writes: 3 s on an HDD at 10 ms each, 30 ms on an SSD at 100 μs, against 2 ms for one log append. No-force is 15 to 1,500 times cheaper per commit, which is why every production system chooses it.

After a power cut. At restart, recovery reads the WAL from the last checkpoint's position. *Committed transaction* (its commit record is in the log): every one of its changes whose page was not yet written is **redone** from the log onto the data page; nothing committed is lost, which is the promise of durability. *Uncommitted transaction* (no commit record): it is treated as aborted. Any of its pages that had already been written (steal) must not remain visible; in a classic ARIES system they are **undone** from the log's old values, and in PostgreSQL they need no undo at all because MVCC never overwrote the old row versions: the transaction's new tuple versions carry a transaction id whose status is "aborted", so no query ever sees them, and `VACUUM` reclaims the space later. The end state is the same in both designs: committed work present, uncommitted work absent, and the 2 ms commit was safe because the log, not the pages, is what commit waits for.

Where it is in the textbooks

D1: Chapter 6, §7.4 (sizing `shared_buffers`: the three failures of 80%), §5.4 (working sets and diminishing returns), §7.3 (what to measure); Chapter 4, §7.2 (`work_mem` is per operator per connection). D2: Chapter 4, §5.2 (index nested loop), §5.3 (sort-merge: sorted inputs, sorted output, inequality joins), §5.5 (which join wins) and §7.2 (the `enable_*` switches are penalties); Chapter 3, §3.2 (the index descent that makes the nested loop cheap). D3: Chapter 6, §6.1–6.3 (dirty pages, steal/force, the write-ahead rule and the 300-page worked example) and its MVCC margin note; Chapter 2, §6.2 (why the DBMS must control write ordering).

Coverage summary

Every point of the mock is covered by the six chapters. Chapters 1 to 5 carry A1–A6, B1, B3, C1, C2, C4, C5 and D2 (76 points); Chapter 6 carries A7, A8, C3, D1, D3 and half of B2 (24 points), and C3's ticket is Chapter 6's own worked example. The formula sheet's buffer-pool line (clock-sweep, `usage_count` 0–5, a 256 KB ring above `shared_buffers`/4) is Chapter 6 §4.3 and §5.3. The review's five errors all appear here: rows versus pages (C4), read versus disk (A7, C3), "an index always helps" (C4), "hash join is always best" (B3, D2), the first sort pass (A5, B1).